

# 第三章 最小二乘问题的解法

## 第二节 初等正交变换

张振宇

上海财经大学 数学学院

September 1, 2025

## 对 Gauss 变换的回顾

直接对正则化方程求解，其条件数是原问题的平方，故需要给出更实用的最小二乘问题的求解算法——正交化方法。为此，需要先了解矩阵的初等正交变换。为了便于理解，先回顾 Gauss 变换。

## 对Gauss变换的回顾

直接对正则化方程求解，其条件数是原问题的平方，故需要给出更实用的最小二乘问题的求解算法——正交化方法。为此，需要先了解矩阵的初等正交变换。为了便于理解，先回顾Gauss变换。

$\forall \mathbf{x} \in \mathbb{R}^n$ ,  $\exists$  初等下三角阵  $L \in \mathbb{R}^{n \times n}$  和  $\alpha \in \mathbb{R}$ , 使得  $L\mathbf{x} = \alpha \mathbf{e}_1$ , 这里  $\mathbf{e}_1$  是单位矩阵  $I_n$  的第一列。其中,

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & 0 & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & 0 & 0 & \cdots & 1 \end{pmatrix}.$$

## 对Gauss变换的回顾

直接对正则化方程求解，其条件数是原问题的平方，故需要给出更实用的最小二乘问题的求解算法——正交化方法。为此，需要先了解矩阵的初等正交变换。为了便于理解，先回顾Gauss变换。

$\forall \mathbf{x} \in \mathbb{R}^n, \exists$  初等下三角阵  $L \in \mathbb{R}^{n \times n}$  和  $\alpha \in \mathbb{R}$ ，使得  $L\mathbf{x} = \alpha \mathbf{e}_1$ ，这里  $\mathbf{e}_1$  是单位矩阵  $I_n$  的第一列。其中，

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & 0 & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & 0 & 0 & \cdots & 1 \end{pmatrix}.$$

而用初等正交变换对矩阵进行上三角化的过程与之类似，唯一不同的是变换由一系列初等正交矩阵形成。

## 镜像变换定义

定义3.2.1 Householder变换（镜像变换）

设 $\boldsymbol{w} \in \mathbb{R}^n$ 满足 $\|\boldsymbol{w}\|_2 = 1$ , 定义 $H \in \mathbb{R}^{n \times n}$ 为

$$H = I - 2\boldsymbol{w}\boldsymbol{w}^T,$$

则称 $H$ 为Householder变换或者初等反射矩阵或镜像变换。

## Householder变换性质

### 定理3.2.1 Householder变换的性质

设 $H$ 是Householder变换矩阵，那么 $H$ 满足：

- ① 对称性：  $H^T = H$ ;

## Householder变换性质

### 定理3.2.1 Householder变换的性质

设 $H$ 是Householder变换矩阵，那么 $H$ 满足：

- ① 对称性：  $H^T = H$ ;
- ② 正交性：  $H^T H = I$ ;

## Householder变换性质

### 定理3.2.1 Householder变换的性质

设 $H$ 是Householder变换矩阵，那么 $H$ 满足：

- ① 对称性：  $H^T = H$ ;
- ② 正交性：  $H^T H = I$ ;
- ③ 对合性：  $H^2 = I$  （或者  $(I - H)(I + H) = 0$ ）；



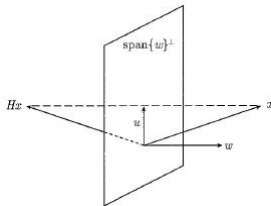
## Householder变换性质

### 定理3.2.1 Householder变换的性质

设 $H$ 是Householder变换矩阵，那么 $H$ 满足：

- ① 对称性：  $H^T = H$ ;
- ② 正交性：  $H^T H = I$ ;
- ③ 对合性：  $H^2 = I$  （或者  $(I - H)(I + H) = 0$ ）；
- ④ 反射性：  $\forall x \in \mathbb{R}^n$ ,  $Hx$ 是 $x$ 关于 $w$ 的垂直超平面 $\text{span}\{w\}^\perp$ 的镜像反射。

## Householder变换反射性证明

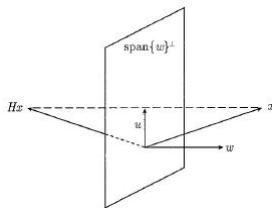


设  $x \in \mathbb{R}^n$ , 则  $x = u + \alpha w$ , 其中  $u \in \text{span}\{w\}^\perp, \alpha \in \mathbb{R}$ 。注意到:  $u^T w = 0, w^T w = 1$ , 于是

$$\begin{aligned} Hx &= (I - 2ww^T)(u + \alpha w) \\ &= u + \alpha w - 2ww^T u - 2\alpha ww^T w \\ &= u - \alpha w. \end{aligned}$$

即  $Hx$  是  $x$  关于  $\text{span}\{w\}^\perp$  的镜像反射。

## Householder变换反射性证明



设  $x \in \mathbb{R}^n$ , 则  $x = u + \alpha w$ , 其中  $u \in \text{span}\{w\}$ ,  $\alpha \in \mathbb{R}$ 。注意到:  $u^T w = 0$ ,  $w^T w = 1$ , 于是

$$\begin{aligned} Hx &= (I - 2ww^T)(u + \alpha w) \\ &= u + \alpha w - 2ww^T u - 2\alpha ww^T w \\ &= u - \alpha w. \end{aligned}$$

即  $Hx$  是  $x$  关于  $\text{span}\{w\}^\perp$  的镜像反射。特别  $w$  是平行于  $x - Hx$  的单位向量  $\frac{x - Hx}{\|x - Hx\|}$ 。

## 使用Householder变换把给定向量的指定分量消成零

### 定理3.2.2

设  $\mathbf{0} \neq \mathbf{x} \in \mathbb{R}^n$ , 则可构造单位向量  $\mathbf{w} \in \mathbb{R}^n$ , 使得Householder变换矩阵  $H = I - 2\mathbf{w}\mathbf{w}^T$  满足

$$H\mathbf{x} = \alpha\mathbf{e}_1,$$

其中  $\alpha = \pm\|\mathbf{x}\|_2$ .

## 使用Householder变换把给定向量的指定分量消成零

### 定理3.2.2

设  $\mathbf{0} \neq \mathbf{x} \in \mathbb{R}^n$ , 则可构造单位向量  $\mathbf{w} \in \mathbb{R}^n$ , 使得Householder变换矩阵  $H = I - 2\mathbf{w}\mathbf{w}^T$  满足

$$H\mathbf{x} = \alpha\mathbf{e}_1,$$

其中  $\alpha = \pm\|\mathbf{x}\|_2$ .

通过推导可知:

$$H\mathbf{x} = (I - 2\mathbf{w}\mathbf{w}^T)\mathbf{x} = \mathbf{x} - 2(\mathbf{w}^T\mathbf{x})\mathbf{w} = \alpha\mathbf{e}_1 \Rightarrow \mathbf{w} = \frac{\mathbf{x} - \alpha\mathbf{e}_1}{\|\mathbf{x} - \alpha\mathbf{e}_1\|_2} \stackrel{\text{def}}{=} \frac{\mathbf{u}}{\|\mathbf{u}\|_2}.$$

## 使用Householder变换把给定向量的指定分量消成零

### 定理3.2.2

设  $\mathbf{0} \neq \mathbf{x} \in \mathbb{R}^n$ , 则可构造单位向量  $\mathbf{w} \in \mathbb{R}^n$ , 使得Householder变换矩阵  $H = I - 2\mathbf{w}\mathbf{w}^T$  满足

$$H\mathbf{x} = \alpha\mathbf{e}_1,$$

其中  $\alpha = \pm\|\mathbf{x}\|_2$ .

通过推导可知:

$$H\mathbf{x} = (I - 2\mathbf{w}\mathbf{w}^T)\mathbf{x} = \mathbf{x} - 2(\mathbf{w}^T\mathbf{x})\mathbf{w} = \alpha\mathbf{e}_1 \Rightarrow \mathbf{w} = \frac{\mathbf{x} - \alpha\mathbf{e}_1}{\|\mathbf{x} - \alpha\mathbf{e}_1\|_2} \stackrel{\text{def}}{=} \frac{\mathbf{u}}{\|\mathbf{u}\|_2}.$$

$\mathbf{w} = \frac{1}{2\mathbf{w}^T\mathbf{x}}(\mathbf{x} - \alpha\mathbf{e}_1) \Rightarrow \mathbf{w}$  是  $\mathbf{x}, \mathbf{e}_1$  的线性组合. 而  $|\alpha| = \|H\mathbf{x}\|_2 = \|\mathbf{x}\|_2 \Rightarrow \mathbf{u} = \mathbf{x} \pm \|\mathbf{x}\|_2\mathbf{e}_1$ .

为了保证计算  $\mathbf{u}$  的第一个分量时, 不出现两个相近的数相减而导致有效数字丢失, 一般应

取  $\alpha = -\text{sign}(x_1)\|\mathbf{x}\|_2$ , 即  $\mathbf{u} = \mathbf{x} + \text{sign}(x_1)\|\mathbf{x}\|_2\mathbf{e}_1$ , 其中  $x_1$  为  $\mathbf{x}$  的第一个分量.

## Householder变换算法推导

- ① 还可以有另一种避免有效数字丢失的方法：若取 $\alpha > 0$ , 记 $\mathbf{u} = (u_1, \dots, u_n)^T$ , 即取

$$u_1 = x_1 - \|\mathbf{x}\|_2 = \frac{x_1^2 - \|\mathbf{x}\|_2^2}{x_1 + \|\mathbf{x}\|_2} = \frac{-(x_2^2 + \dots + x_n^2)}{x_1 + \|\mathbf{x}\|_2}.$$

只要 $x_1 > 0$ , 便可避免相近数相减。

## Householder变换算法推导

- ① 还可以有另一种避免有效数字丢失的方法：若取 $\alpha > 0$ , 记 $\mathbf{u} = (u_1, \dots, u_n)^T$ , 即取

$$u_1 = x_1 - \|\mathbf{x}\|_2 = \frac{x_1^2 - \|\mathbf{x}\|_2^2}{x_1 + \|\mathbf{x}\|_2} = \frac{-(x_2^2 + \dots + x_n^2)}{x_1 + \|\mathbf{x}\|_2}.$$

只要 $x_1 > 0$ , 便可避免相近数相减。

- ② 由于

$$H = I - 2\mathbf{w}\mathbf{w}^T = I - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T \stackrel{\text{def}}{=} I - \beta\mathbf{u}\mathbf{u}^T,$$

因此实际计算时没有必要计算出 $\mathbf{w}$ . 实际计算时, Householder变换矩阵往往被用来与向量相乘, 因此计算出 $\beta$ 和 $\mathbf{u}$ 以后, 仅需计算一次向量-向量乘积和加法运算, 便可以完成Householder变换矩阵与向量乘法。



## Householder变换算法推导

- ① 还可以有另一种避免有效数字丢失的方法：若取 $\alpha > 0$ , 记 $\mathbf{u} = (u_1, \dots, u_n)^T$ , 即取

$$u_1 = x_1 - \|\mathbf{x}\|_2 = \frac{x_1^2 - \|\mathbf{x}\|_2^2}{x_1 + \|\mathbf{x}\|_2} = \frac{-(x_2^2 + \dots + x_n^2)}{x_1 + \|\mathbf{x}\|_2}.$$

只要 $x_1 > 0$ , 便可避免相近数相减。

- ② 由于

$$H = I - 2\mathbf{w}\mathbf{w}^T = I - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T \stackrel{\text{def}}{=} I - \beta\mathbf{u}\mathbf{u}^T,$$

因此实际计算时没有必要计算出 $\mathbf{w}$ . 实际计算时, Householder变换矩阵往往被用来与向量相乘, 因此计算出 $\beta$ 和 $\mathbf{u}$ 以后, 仅需计算一次向量-向量乘积和加法运算, 便可以完成Householder变换矩阵与向量乘法。

- ③ 实际计算时会把 $u_1$ 化成1, 这样只需保存 $\mathbf{u}$ 的后 $n - 1$ 分量到 $\mathbf{x}$ 的相应位置,  $u_1$ 因等于1就无需保存了。

# Householder变换算法推导

- ① 还可以有另一种避免有效数字丢失的方法：若取 $\alpha > 0$ , 记 $\mathbf{u} = (u_1, \dots, u_n)^T$ , 即取

$$u_1 = x_1 - \|\mathbf{x}\|_2 = \frac{x_1^2 - \|\mathbf{x}\|_2^2}{x_1 + \|\mathbf{x}\|_2} = \frac{-(x_2^2 + \dots + x_n^2)}{x_1 + \|\mathbf{x}\|_2}.$$

只要 $x_1 > 0$ , 便可避免相近数相减。

- ② 由于

$$H = I - 2\mathbf{w}\mathbf{w}^T = I - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T \stackrel{\text{def}}{=} I - \beta\mathbf{u}\mathbf{u}^T,$$

因此实际计算时没有必要计算出 $\mathbf{w}$ . 实际计算时, Householder变换矩阵往往被用来与向量相乘, 因此计算出 $\beta$ 和 $\mathbf{u}$ 以后, 仅需计算一次向量-向量乘积和加法运算, 便可以完成Householder变换矩阵与向量乘法。

- ③ 实际计算时会把 $u_1$ 化成1, 这样只需保存 $\mathbf{u}$ 的后 $n - 1$ 分量到 $\mathbf{x}$ 的相应位置,  $u_1$ 因等于1就无需保存了。
- ④ ①中需要计算 $x_i^2 (i = 2, \dots, n)$ , 当某个 $|x_i|$ 很大时, 其平方可能会上溢出。为了避免出现这种情况, 可以规范化 $\mathbf{x}$ 来计算, 即用 $\mathbf{x}/\|\mathbf{x}\|_\infty$ 来替代 $\mathbf{x}$ 构造 $\mathbf{u}$ 。

# 计算Householder变换算法

## 算法3.2.1 计算Householder变换

```
1 function [u, beta]=House(x)
2     n=length(x); eta=norm(x,inf); x=x/eta;
3     sigma=x(2:n)'*x(2:n); u=x;
4     if sigma==0
5         beta = 0;
6     else
7         alpha=sqrt(x(1)*x(1)+sigma);
8         if x(1)<=0
9             u(1)=x(1)-alpha;
10        else
11            u(1)=-sigma/(x(1)+alpha);
12        end
13        beta=2*u(1)*u(1)/(sigma+u(1)*u(1)); u=u/u(1);
14    end
```

## 利用Householder变换消去一个向量的部分分量为零

如果想把 $\boldsymbol{x} \in \mathbb{R}^n$ 的分量:  $x_{k+1}, \dots, x_j$ 消去成零, 只需定义

$$\boldsymbol{u} = (0 \quad \cdots \quad 0 \quad x_k - \alpha \quad x_{k+1} \quad \cdots \quad x_j \quad 0 \quad \cdots \quad 0)^T$$

其中,  $\alpha^2 = \sum_{i=k}^j x_i^2$ .

## 利用Householder变换约化矩阵

与利用Householder变换约化向量一样，不必求出变换矩阵 $H$ ，只需利用构造 $H$ 的 $\beta$ 和 $\mathbf{u}$ ，使用很少的计算量，便可完成计算。 $\forall A \in \mathbb{R}^{m \times n}$ ,

$$HA = (I - \beta \mathbf{u} \mathbf{u}^T) A = A - \beta \mathbf{u} (A^T \mathbf{u})^T = A - \mathbf{u} \mathbf{v}^T,$$

其中， $\mathbf{v} = \beta A^T \mathbf{u}$ .

## 利用Householder变换约化矩阵

与利用Householder变换约化向量一样，不必求出变换矩阵 $H$ ，只需利用构造 $H$ 的 $\beta$ 和 $\mathbf{u}$ ，使用很少的计算量，便可完成计算。 $\forall A \in \mathbb{R}^{m \times n}$ ,

$$HA = (I - \beta \mathbf{u} \mathbf{u}^T) A = A - \beta \mathbf{u} (A^T \mathbf{u})^T = A - \mathbf{u} \mathbf{v}^T,$$

其中， $\mathbf{v} = \beta A^T \mathbf{u}$ . 具体算法如下:

- ① 计算 $\mathbf{v} = \beta A^T \mathbf{u}$ ;

## 利用Householder变换约化矩阵

与利用Householder变换约化向量一样，不必求出变换矩阵 $H$ ，只需利用构造 $H$ 的 $\beta$ 和 $\mathbf{u}$ ，使用很少的计算量，便可完成计算。 $\forall A \in \mathbb{R}^{m \times n}$ ,

$$HA = (I - \beta \mathbf{u} \mathbf{u}^T) A = A - \beta \mathbf{u} (A^T \mathbf{u})^T = A - \mathbf{u} \mathbf{v}^T,$$

其中， $\mathbf{v} = \beta A^T \mathbf{u}$ . 具体算法如下:

- ① 计算 $\mathbf{v} = \beta A^T \mathbf{u}$ ;
- ② 计算 $HA = B = A - \mathbf{u} \mathbf{v}^T$ .

## 利用Householder变换约化矩阵

与利用Householder变换约化向量一样，不必求出变换矩阵 $H$ ，只需利用构造 $H$ 的 $\beta$ 和 $\mathbf{u}$ ，使用很少的计算量，便可完成计算。 $\forall A \in \mathbb{R}^{m \times n}$ ,

$$HA = (I - \beta \mathbf{u} \mathbf{u}^T) A = A - \beta \mathbf{u} (A^T \mathbf{u})^T = A - \mathbf{u} \mathbf{v}^T,$$

其中， $\mathbf{v} = \beta A^T \mathbf{u}$ . 具体算法如下:

- ① 计算 $\mathbf{v} = \beta A^T \mathbf{u}$ ;
- ② 计算 $HA = B = A - \mathbf{u} \mathbf{v}^T$ .

整个计算量为 $4mn$ .



## Givens变换

利用Givens变换可以把向量中的某个分量化为零。Givens变换矩阵定义如下：

$$G(i, k, \theta) = I + s(\mathbf{e}_i \mathbf{e}_k^T - \mathbf{e}_k \mathbf{e}_i^T) + (c - 1)(\mathbf{e}_i \mathbf{e}_i^T + \mathbf{e}_k \mathbf{e}_k^T) =$$
$$\begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & c & \cdots & s & \\ & & \vdots & \ddots & \vdots & \\ & & -s & \cdots & c & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix}.$$

其中,  $c = \cos \theta$ ,  $s = \sin \theta$ .

## Givens变换矩阵是正交阵

记

$$B = G^T G = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & c & \cdots & -s & \\ & & \vdots & \ddots & \vdots & \\ & & s & \cdots & c & \\ & & & & & \ddots \\ & & & & & & 1 \end{pmatrix} \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & c & \cdots & s & \\ & & \vdots & \ddots & \vdots & \\ & & -s & \cdots & c & \\ & & & & & \ddots \\ & & & & & & 1 \end{pmatrix},$$

则除了 $b_{ii}, b_{ik}, b_{ki}, b_{kk}$ 以外, 其余 $b_{jl} = e_{jl}$ ,  $e_{jl}$ 是单位矩阵的元素。而

$$b_{ii} = b_{kk} = s^2 + c^2 = \sin^2 \theta + \cos^2 \theta = 1, b_{ik} = b_{ki} = -cs + cs = 0 \Rightarrow B = I.$$

## 利用Givens变换约化向量

设 $\boldsymbol{x} \in \mathbb{R}^n$ , 令 $\boldsymbol{y} = G(i, k, \theta)\boldsymbol{x}$ , 则

$$y_i = cx_i + sx_k,$$

$$y_k = -sx_i + cx_k,$$

$$y_j = x_j, \quad j \neq i, k$$

因此, 若要约化 $y_k = 0$ , 只需

$$c = \frac{x_i}{\sqrt{x_i^2 + x_k^2}}, \quad s = \frac{x_k}{\sqrt{x_i^2 + x_k^2}},$$

便有

$$y_i = \sqrt{x_i^2 + x_k^2}, \quad y_k = 0.$$

从几何上看,  $G(i, k, \theta)\boldsymbol{x}$ 是在 $(i, k)$ 坐标面内将 $\boldsymbol{x}$ 按顺时针方向做了 $\theta$ 弧度的旋转, 所以Givens变换又被称为平面旋转变换。

## 计算Givens变换的算法

从前面的推导可以看出计算 $\mathbf{y} = G(i, k, \theta)\mathbf{x}$ 只用到 $c, s, x_i, x_k$ , 且仅影响到 $y_i, y_k$ , 因此我们可以简化一次Givens旋转变换为

$$\begin{pmatrix} y_i \\ y_k \end{pmatrix} = \begin{pmatrix} c & s \\ -s & c \end{pmatrix} \begin{pmatrix} x_i \\ x_k \end{pmatrix} = \begin{pmatrix} r \\ 0 \end{pmatrix}.$$

于是仅需给出 $x_i, x_k$ 便可计算得到构造Givens变换矩阵的 $c$ 和 $s$ .

若记 $a = x_i, b = x_k$ , 则为了避免计算 $c, s$ 时产生上溢出, 可以使用以下的等价计算方法

- 若 $|b| > |a|$ , 则 $\tau = \frac{a}{b}, s = \frac{1}{\sqrt{1+\tau^2}}, c = s\tau$ ;
- 若 $|b| \leq |a|$ , 则 $\tau = \frac{b}{a}, c = \frac{1}{\sqrt{1+\tau^2}}, s = s\tau$ .

具体算法如下:

## 计算Givens变换的算法.

### 计算Givens变换

```
1 function [c, s]=Givens(a, b)
2     if b==0
3         c=1; s=0;
4     else
5         if abs(b)>abs(a)
6             tau=a/b; s=1/sqrt(1+tau*tau); c=s*tau;
7         else
8             tau=b/a; c=1/sqrt(1+tau*tau); s=c*tau;
9         end
10    end
```

无论计算Householder变换，还是Givens变换，其数值计算过程都十分稳定。

## 使用 Givens 变换约化矩阵

设  $A \in \mathbb{R}^{n \times q}$ , 则  $B = G(i, k, \theta)A = (b_{st}) \in \mathbb{R}^{n \times q}$ . 其中

$$b_{st} = \begin{cases} ca_{it} + sa_{kt}, & s = i; \\ -sa_{it} + ca_{kt}, & s = k; \\ a_{st}, & s \neq i, k. \end{cases}, \quad s = 1, \dots, n; t = 1, \dots, q.$$

Givens 变换矩阵右乘  $A$  的算法完全类似, 请同学们自行推导。